ASSIGNMENT – 11.2

2303A51091
Batch : 29

Task Description -1 – (Stack Using AI Guidance)
• Task: With the help of AI, design and implement a Stack data structure supporting basic stack operations.
Expected Output:
• A Python Stack class supporting push, pop, peek, and empty-Check operations with proper documentation.

task1.py > Queue > size

```python
from collections import deque


class Queue:
    """
    A Queue data structure that follows FIFO (First In First Out) principle.
    Uses collections.deque for efficient O(1) operations.
    """

    def __init__(self):
        """Initialize an empty queue."""
        self.items = deque()

    def enqueue(self, element):
        """
        Add an element to the rear of the queue.

        Args:
            element: The element to add to the queue.
        """
        self.items.append(element)

    def dequeue(self):
        """
        Remove and return the front element from the queue.

        Returns:
            The front element, or None if the queue is empty.
        """
        if not self.is_empty():
            return self.items.popleft()
        return None

    def front(self):
        """
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    Python

arepallyramcharan@Arepallys-MacBook-Air assignment - 11.2.py % /usr/bin/python3 "/Users/arepallyramcharan/ai ass
istent.py/assignment - 11.2.py/task1.py"
Queue size: 3
Front element: 10
Dequeued: 10
Dequeued: 20
Queue size: 1
Front of empty queue: None
arepallyramcharan@Arepallys-MacBook-Air assignment - 11.2.py %
```

Observation:

1. AI assistance helps in quickly designing the stack structure and operations.
2. The stack follows the **LIFO (Last In First Out)** principle.
3. Operations like push, pop, peek, and empty-check ensure proper stack functionality.
4. Documentation improves code readability and understanding.
5. AI-generated implementations help beginners understand data structure concepts efficiently.

Task Description -2 – (Queue Design)
• Task:Use AI assistance to create a Queue data structure following FIFO principles
Expected Output:
• A complete Queue implementation including enqueue, dequeue, front element access, and size calculation

task2.py > ...

```python


class Queue:
    """
    A Queue data structure that follows FIFO (First In First Out) principle.
    """

    def __init__(self):
        """Initialize an empty queue."""
        self.items = []

    def enqueue(self, element):
        """
        Add an element to the rear of the queue.

        Args:
            element: The element to add to the queue.
        """
        self.items.append(element)

    def dequeue(self):
        """
        Remove and return the front element from the queue.

        Returns:
            The front element, or None if the queue is empty.
        """
        if not self.is_empty():
            return self.items.pop(0)
        return None

    def front(self):
        """
        Return the front element of the queue without removing it.

```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment - 11.2.py % /usr/bin/python3 "/Users/arepallyramcharan/ai ass
istent.py/assignment - 11.2.py/task2.py"
Queue size: 3
Front element: 10
Dequeued: 10
Dequeued: 20
Queue size: 1
Front of empty queue: None
arepallyramcharan@Arepallys-MacBook-Air assignment - 11.2.py %
Ln 80, Col 1    Spaces: 4    UTF-8
```

Observation:

- The queue data structure follows the **FIFO (First In First Out)** principle.
- AI guidance simplifies the implementation of enqueue and dequeue operations.
- Front element access helps identify the next element to be removed.
- Size calculation ensures proper monitoring of queue elements.
- AI-assisted development speeds up coding while maintaining correctness.

Task Description -3 –(Singly Linked List Construction)
• Task: Utilize AI to build a singly linked list supporting insertion and traversal.
Expected Output:
• Correctly functioning linked list with node creation, insertion logic, and display functionality.

task3.py > ...

```python
# Singly Linked List Implementation in Python

class Node:
    """A node in the linked list"""
    def __init__(self, data):
        self.data = data  # Store the data
        self.next = None  # Reference to the next node


class SinglyLinkedList:
    """A singly linked list implementation"""
    def __init__(self):
        self.head = None  # Initialize an empty list

    def insert_at_beginning(self, data):
        """Insert a node at the beginning of the list"""
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def insert_at_end(self, data):
        """Insert a node at the end of the list"""
        new_node = Node(data)

        # If list is empty, make new node the head
        if not self.head:
            self.head = new_node
            return

        # Traverse to the last node
        current = self.head
        while current.next:
            current = current.next
        current.next = new_node

```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment - 11.2.py % /usr/bin/python3 "/Users/arepallyramcharan/ai ass
istent.py/assignment - 11.2.py/task3.py"
10 -> 20 -> 30 -> None

After inserting 5 at beginning:
5 -> 10 -> 20 -> 30 -> None

After inserting 15 at position 2:
5 -> 10 -> 15 -> 20 -> 30 -> None
arepallyramcharan@Arepallys-MacBook-Air assignment - 11.2.py %
```

Ln 89, Col 27 Spaces: 4 UTF-8

Observation:

- A singly linked list consists of nodes connected through references.
- Each node stores data and a pointer to the next node.
- Insertion logic allows adding elements dynamically.
- Traversal helps display or process all nodes in the list.
- AI support makes understanding linked list structure easier for learners.

Task Description -4 – (Binary Search Tree Operations)

• Task: Implement a Binary Search Tree with AI support focusing on insertion and traversal.

Expected Output:

• BST program with correct node insertion and in-order traversal output.

task4.py > BinarySearchTree

```python
class Node:
    """
    Represents a single node in the Binary Search Tree.

    Attributes:
        data: The value stored in the node.
        left: Reference to the left child node.
        right: Reference to the right child node.
    """
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BinarySearchTree:
    """
    A Binary Search Tree implementation that maintains ordered data.

    Attributes:
        root: The root node of the tree.
    """
    def __init__(self):
        self.root = None

    def insert(self, value):
        """
        Insert a new value into the BST following BST rules.

        BST Rule: Values less than node go left, greater values go right.

        Args:
            value: The value to insert into the tree.
        """
        if self.root is None:
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment - 11.2.py % /usr/bin/python3 "/Users/arepallyramcharan/ai ass
istent.py/assignment - 11.2.py/task4.py"
Inserting values: [50, 30, 70, 20, 40, 60, 80]
In-order traversal: [20, 30, 40, 50, 60, 70, 80]
arepallyramcharan@Arepallys-MacBook-Air assignment - 11.2.py %
```

Observation:

1. Binary Search Tree (BST) organizes data in a hierarchical structure.
2. Node insertion follows the BST property (left < root < right).
3. In-order traversal prints elements in sorted order.
4. AI assistance helps implement tree operations correctly.
5. Proper traversal techniques improve understanding of tree structures.

Task Description -5 – (Hash Table Implementation)
• Task: Create a hash table using AI with collision handling
Expected Output:
• Hash table supporting insert, search, and delete using chaining or open

task5.py > ...

```python
class HashTable:
    """Hash Table implementation using chaining for collision handling."""

    def __init__(self, size=10):
        """Initialize hash table with given size."""
        self.size = size
        # Create a list of empty lists for chaining
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Generate hash value for a key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert a key-value pair into the hash table."""
        index = self._hash(key)
        bucket = self.table[index]

        # Check if key already exists and update it
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                return

        # Add new key-value pair to the bucket
        bucket.append((key, value))

    def search(self, key):
        """Search for a key and return its value, or None if not found."""
        index = self._hash(key)
        bucket = self.table[index]

        # Linear search in the bucket
        for k, v in bucket:
            if k == key:
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment - 11.2.py % /usr/bin/python3 "/Users/arepallyramcharan/ai ass
istent.py/assignment - 11.2.py/task5.py"

Deleting 'age'...
Search 'age' after deletion: None

Hash Table after deletion:
Index 1: [('email', 'alice@example.com')]
Index 2: [('name', 'Alice'), ('city', 'New York')]
```

Observation:

1. Hash functions convert keys into index positions.

2. Collision handling techniques like chaining improve reliability.

3. Insert, search, and delete operations provide efficient data access.
4. AI guidance helps implement hashing concepts effectively.
5. A hash table stores data using key-value mapping.